Kaba kuvvet -Brute force
Böl ve yönet-Divide and Conquer
Azalt ve yönet-Decrease and Conquer

Brute force

- it is a straightforward approach to solve a problem, usually directly based on the problem's statement
- it is the easiest (and the most intuitive) way for solving a problem
- algorithms designed by brute force are not always efficient
- The strengths of using a brute force approach are as follows:
- It has wide applicability and is known for its simplicity.
- It yields reasonable algorithms for some important problems such as searching, string matching, and matrix multiplication.
- It yields standard algorithms for simple computational tasks such as sum and product of n numbers, and finding maximum or minimum in a list.
- The weaknesses of the brute force approach are as follows:
- It rarely yields efficient algorithms.
- Some brute force algorithms are unacceptably slow.
- It is neither as constructive nor creative as some other design techniques.

Örnek-brute force: x^n hesabı

- Compute x^n, x is a real number and n a natural numbe

Idea: $x^n = x*x*\ldots*x$ (n times)

```
Power(x,n)
    p←1
    FOR i ← 1,n DO
        p ← p*x
    ENDFOR
    RETURN p
```

Complexity

O(n)

Örnek-brute force: n!

- Compute n!, n a natural number (n>=1)

Idea: n!=1*2*…*n

```
Factorial(n)
    f ← 1
    FOR i ← 1,n DO
        f ← f*i
    ENDFOR
    RETURN f
```

Complexity

O(n)

Örnek-brute force: en büyük ortak bölen

Greatest-Common Divisor

- Given natural numbers m and n (m, n ≥ 1)
- Sequentially check possible divisors

```
t ← min { m, n }
while ( t  > 0 ) do
        if ( m mod t == 0 and n mod t == 0 )
        then return t
        t ← t - 1
```

Örnek: Selection sort

Selection sort

```
i ← 0
while ( i < n – 1 ) do
        min ← i
        j ← i + 1
        while ( j < n ) do
                if ( A[ j ] < A[ min ] )
                then min ← j
                 j ←  j + 1
         if ( i ≠ min )
         then A[ i ] ↔ A[ min ]
         i ← i + 1
```

Selection Sort

- Number of comparisons ?
  - Fixed !
  - $(n – 1) + (n – 2) + \ldots + 2 + 1 = n (n – 1) / 2$
  - $O(n^2)$
- Number of exchanges
  - $B_{Ex}(n) = 0$
  - $W_{Ex}(n) = (n – 1)$ – When does this happen?
  - $A_{Ex}(n) = ?$

Örnek-brute force: String Match

ALGORITHM *BruteForceStringMatch*$(T[0..n-1], P[0..m-1])$

```
//Implements brute-force string matching
//Input: An array T[0..n − 1] of n characters representing a text and
//       an array P[0..m − 1] of m characters representing a pattern
//Output: The index of the first character in the text that starts a
//        matching substring or −1 if the search is unsuccessful
for i ← 0 to n − m do
    j ← 0
    while j < m and P[j] = T[i + j] do
        j ← j + 1
    if j = m return i
return −1
```

Time efficiency: **Θ(mn) comparisons (in the worst case)**

Örnek-brute force: Polinom hesabı

Problem: Find the value of polynomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x^1 + a_0$$

at a point $x = x_0$

Brute-force algorithm

```
p ← 0.0
for i ← n downto 0 do
    power ← 1
        for j ← 1 to i do        //compute x^i
            power ← power * x
        p ← p + a[i] * power
return p
```

Efficiency:

$\Sigma_{0 \le i \le n}\, i = \Theta(n^2)$ multiplications

Polynomial Evaluation: Improvement

We can do better by evaluating from right to left:

Better brute-force algorithm

$p \leftarrow a[0]$
$power \leftarrow 1$
for $i \leftarrow 1$ **to** n **do**
    $power \leftarrow power * x$
    $p \leftarrow p + a[i] * power$
return p

Efficiency:

Θ(*n*) multiplications

Horner's Rule is another linear time method.

Divide and Conquer

- In computer science, **divide and conquer** (**D&C**) is an algorithm design paradigm based on multi-branched recursion.
- A divide and conquer algorithm works by recursively breaking down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly.
- The solutions to the sub-problems are then combined to give a solution to the original problem.

Böl ve yönet Temel yapısı

```
Divide&conquer (n)
IF n<=n_c THEN  <solve P(n) directly to obtain r>
ELSE
    <Divide P(n) in P(n_1), …, P(n_k)>
    FOR i←1,k DO
        r_i ← Divide&conquer(n_i)
    ENDFOR
    r ← Combine (r_1, … r_k)
ENDIF
RETURN r
```

Örnek-divede & conquer: Hanoi Towers

- Divide and conquer only requires reducing the problem to a single smaller problem, such as the classic Tower of Hanoi puzzle, which reduces moving a tower of height *n* to moving a tower of height *n* − 1.

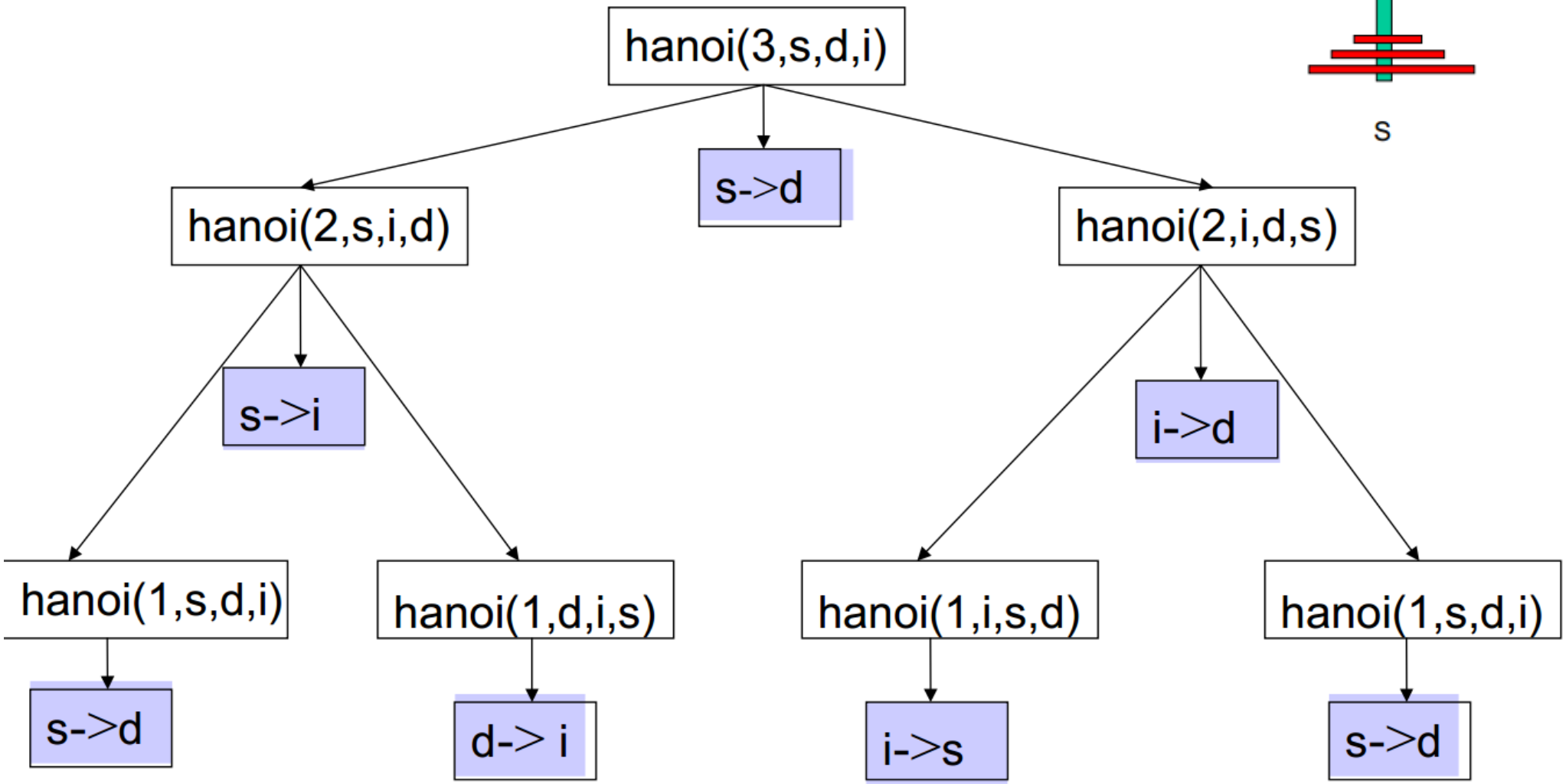
```
Hanoi(n, src, dest, tmp):
    If (n > 0) then
        Hanoi(n − 1, src, tmp, dest)
        Move disk n from src to dest
        Hanoi(n − 1, tmp, dest, src)
```

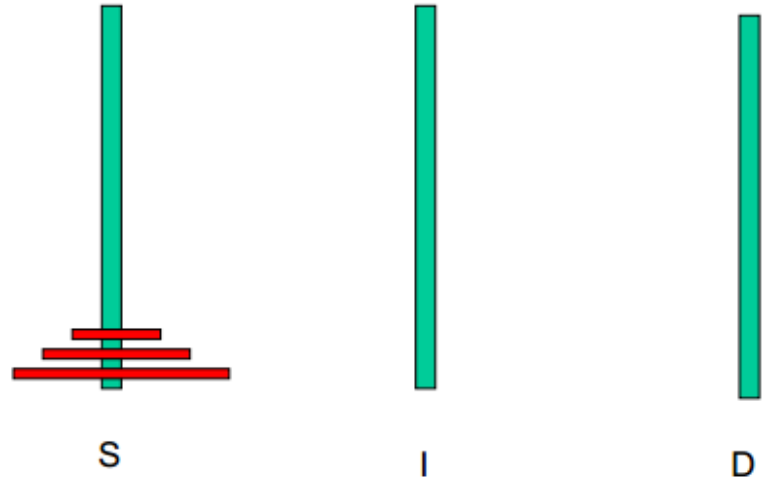
$T(n)$: time to move n disks via recursive strategy

$$T(n) = 2T(n-1) + 1 \qquad n > 1 \qquad \text{and } T(1) = 1$$

Towers of Hanoi

Illustration for n=3.



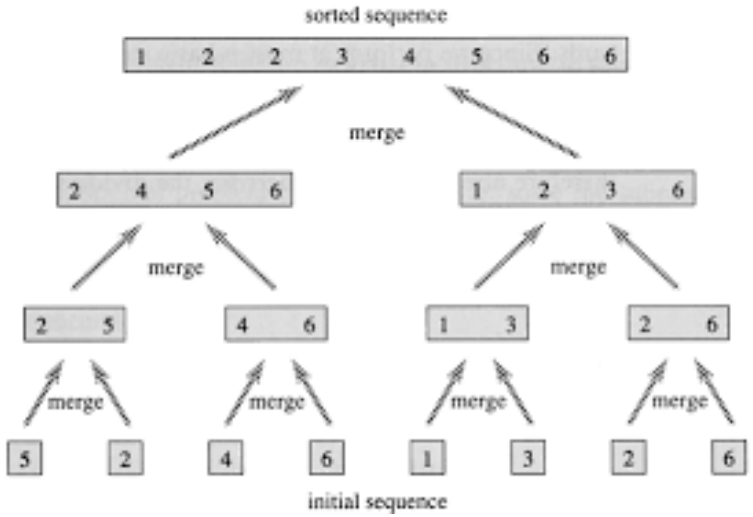


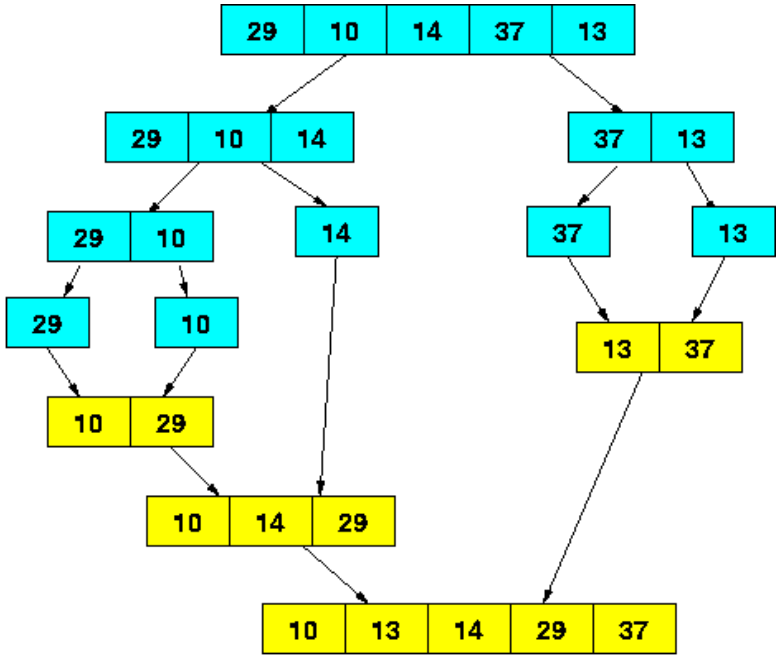
Örnek-divide & conquer: hanoi kuleleri

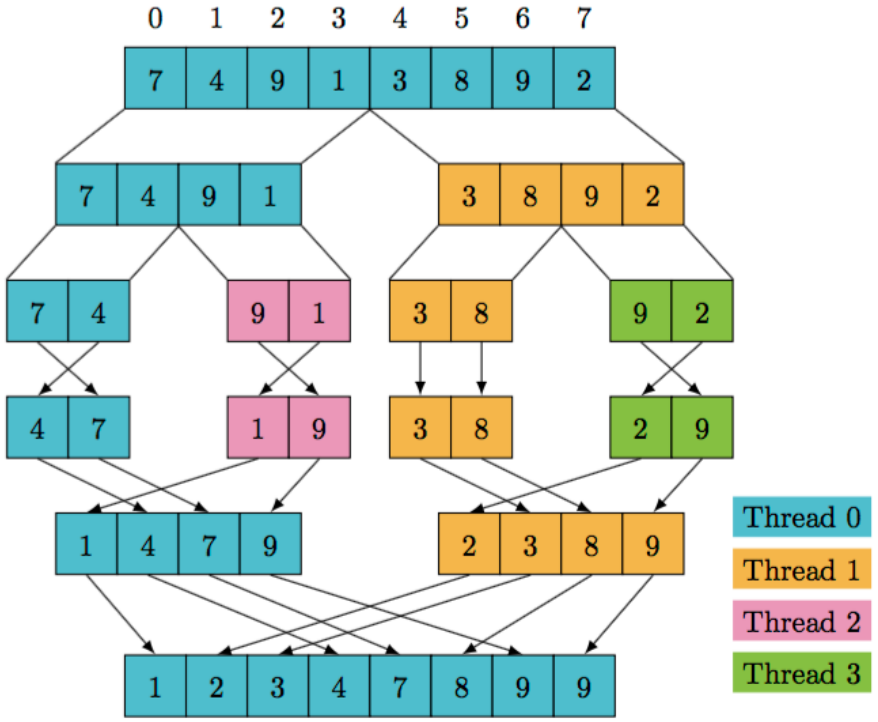
Analysis

$$
\begin{aligned}
T(n) &= 2T(n-1)+1 \\
&= 2^2T(n-2)+2+1 \\
&= \ldots \\
&= 2^iT(n-i)+2^{i-1}+2^{i-2}+\ldots+1 \\
&= \ldots \\
&= 2^{n-1}T(1)+2^{n-2}+\ldots+1 \\
&= 2^{n-1}+2^{n-2}+\ldots+1 \\
&= (2^n-1)/(2-1)=2^n-1
\end{aligned}
$$

Merge sort


sorted sequence
1 2 2 3 4 5 6 6
merge
2 4 5 6
1 2 3 6
merge
merge
2 5
4 6
1 3
2 6
merge
merge
merge
merge
5
2
4
6
1
3
2
6
initial sequence


29 10 14 37 13
29 10 14
37 13
29 10
14
37
13
29
10
13 37
10 29
10 14 29
10 13 14 29 37


0 1 2 3 4 5 6 7
7 4 9 1 3 8 9 2
7 4 9 1
3 8 9 2
7 4
9 1
3 8
9 2
4 7
1 9
3 8
2 9
1 4 7 9
2 3 8 9
1 2 3 4 7 8 9 9
Thread 0
Thread 1
Thread 2
Thread 3

Örnek-divide & conquer: Merge sort

```
MergeSort(array A, int p, int r) {
    if (p < r) {
        q = (p + r)/2
        MergeSort(A, p, q)
        MergeSort(A, q+1, r)
        Merge(A, p, q, r)
    }
}

Merge(array A, int p, int q, int r) {
    array B[p..r]
    i = k = p
    j = q+1
    while (i <= q and j <= r) {
        if (A[i] <= A[j]) B[k++] = A[i++]
        else              B[k++] = A[j++]
    }
    while (i <= q) B[k++] = A[i++]
    while (j <= r) B[k++] = A[j++]
    for i = p to r do A[i] = B[i]
}
```

```
public static void sort(double[] a, double[] tmp )
{
   int width;

   for ( width = 1; width < a.length; width = 2*width )
   {
      // Combine pairs of array a of width "width"
      int i;

      for ( i = 0; i < a.length; i = i + 2*width )
      {
         int left, middle, right;

         left = i;
         middle = i + width;
         right  = i + 2*width;

         merge( a, left, middle, right, tmp );

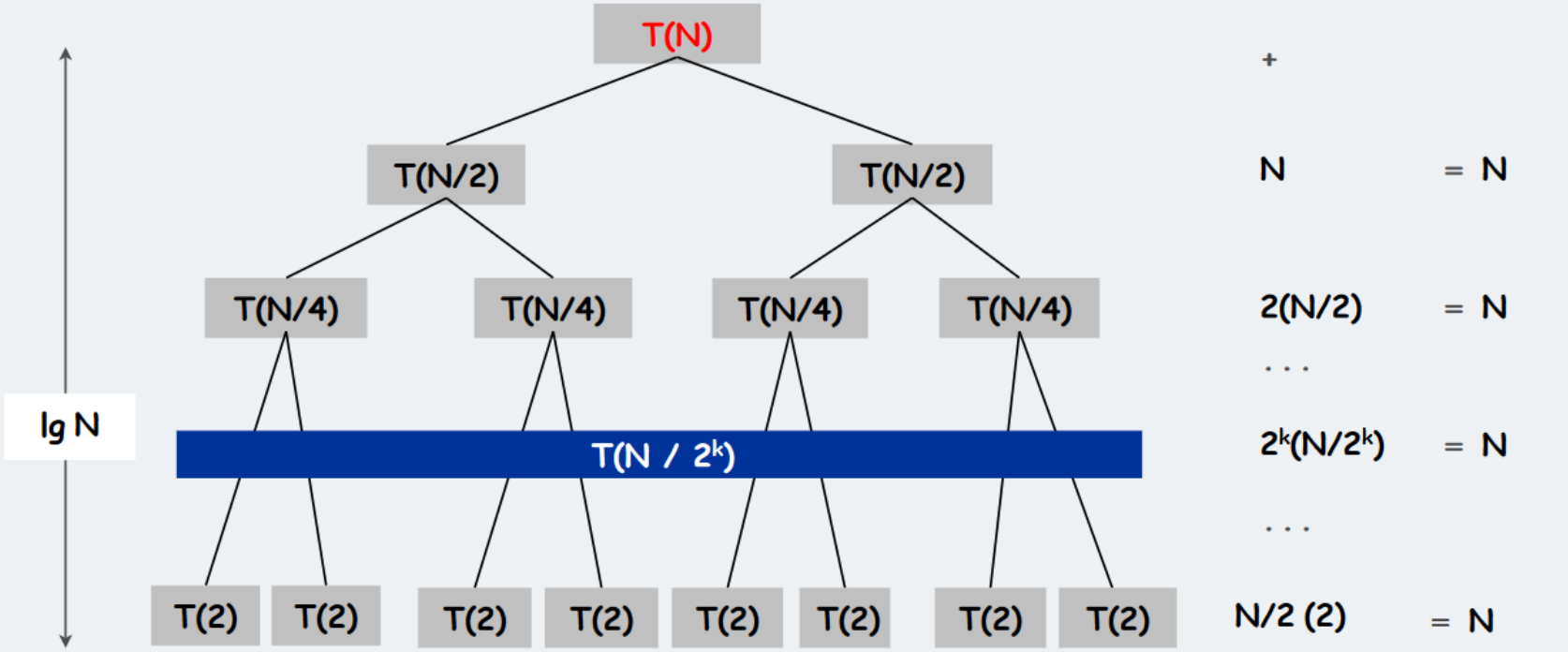
      }
   }
}
```

Örnek-divide & conquer: Merge sort analizi

Mergesort recurrence: Proof 1 (by recursion tree)

$$T(N) = 2\,T(N/2) + N \quad \text{for } N > 1, \text{ with } T(1) = 0$$

(assume that N is a power of 2)



$$T(N) = N \lg N$$

Mergesort recurrence: Proof 2 (by telescoping)

$$T(N) = 2\,T(N/2) + N \quad \text{for } N > 1, \text{ with } T(1) = 0$$

(assume that N is a power of 2)

Pf.

$T(N) = 2\,T(N/2) + N$		given
$T(N)/N = 2\,T(N/2)/N + 1$		divide both sides by N
$= T(N/2)/(N/2) + 1$		algebra
$= T(N/4)/(N/4) + 1 + 1$		telescope (apply to first term)
$= T(N/8)/(N/8) + 1 + 1 + 1$		telescope again
$\ldots$		
$= T(N/N)/(N/N) + 1 + 1 + \ldots + 1$		stop telescoping, T(1) = 0
$= \lg N$		

$$T(N) = N \lg N$$

Mergesort recurrence: Proof 3 (by induction)

$$T(N) = 2\,T(N/2) + N \quad \text{for } N > 1, \text{ with } T(1) = 0$$

(assume that N is a power of 2)

Claim. If T(N) satisfies this recurrence, then $T(N) = N \lg N$.

Pf. [by induction on N]

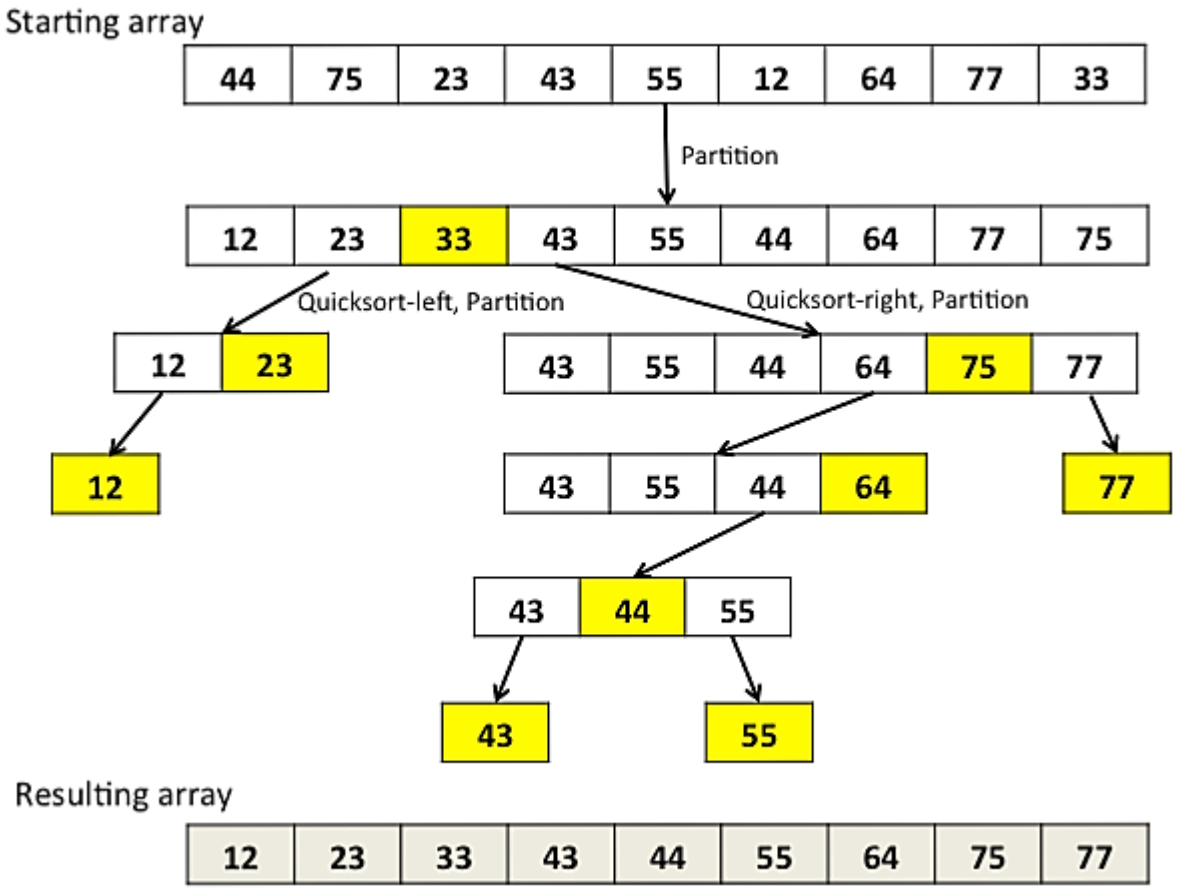
- Base case: N = 1.
- Inductive hypothesis: $T(N) = N \lg N$
- Goal: show that $T(2N) = 2N \lg (2N)$.

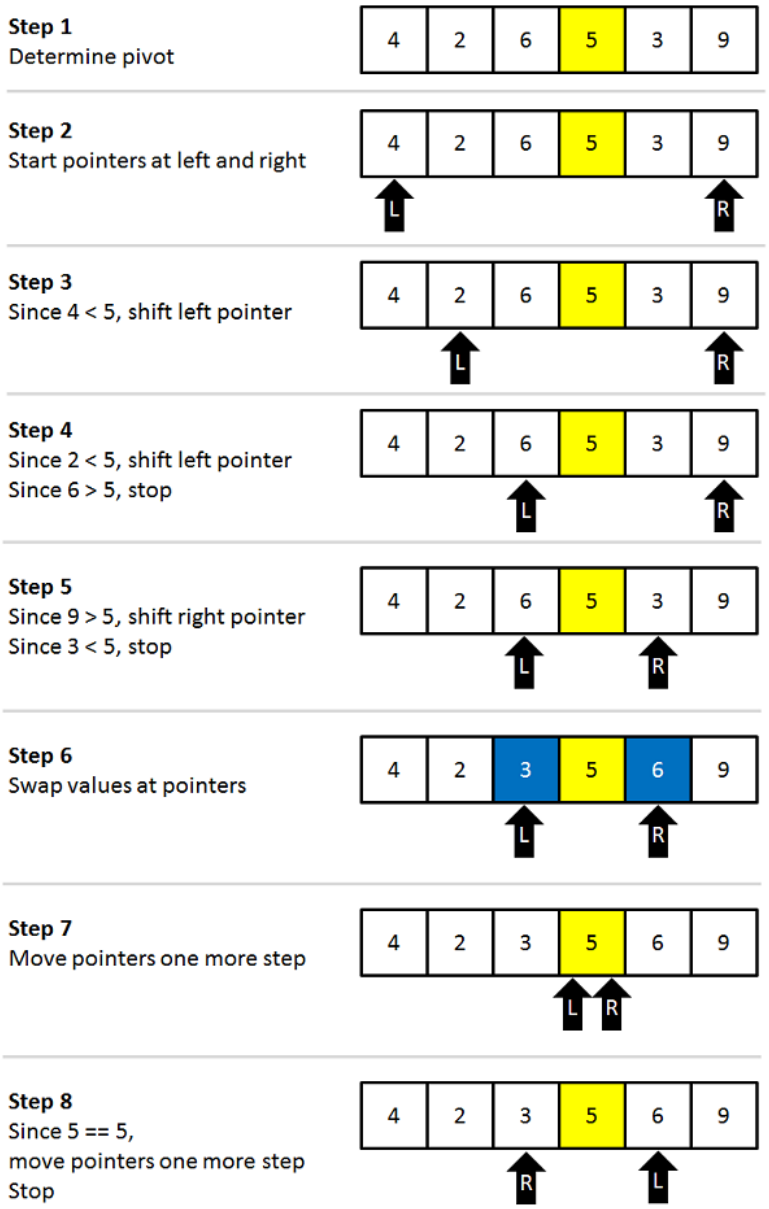
$$\begin{aligned} T(2N) &= 2\,T(N) + 2N && \text{given} \\ &= 2\,N \lg N + 2\,N && \text{inductive hypothesis} \\ &= 2\,N\,(\lg (2N) - 1) + 2N && \text{algebra} \\ &= 2\,N \lg (2N) && \text{QED} \end{aligned}$$

Ex. (for COS 341). Extend to show that $T(N) \sim N \lg N$ for general N

Örnek-divide & conquer: Quick sort

$T(N) = T(i) + T(N - i -1) + cN$ where c is a constant





Örnek-divide & conquer: Quick sort

```
QUICKSORT(A, p, r)
1  if p < r
2      q = PARTITION(A, p, r)
3      QUICKSORT(A, p, q − 1)
4      QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

```
C++
void quickSort(int arr[], int left, int right) {
      int i = left, j = right;
      int tmp;
      int pivot = arr[(left + right) / 2];

      /* partition */
      while (i <= j) {
            while (arr[i] < pivot)
                  i++;
            while (arr[j] > pivot)
                  j--;
            if (i <= j) {
                  tmp = arr[i];
                  arr[i] = arr[j];
                  arr[j] = tmp;
                  i++;
                  j--;
            }
      };

      /* recursion */
      if (left < j)
            quickSort(arr, left, j);
      if (i < right)
            quickSort(arr, i, right);
}
```

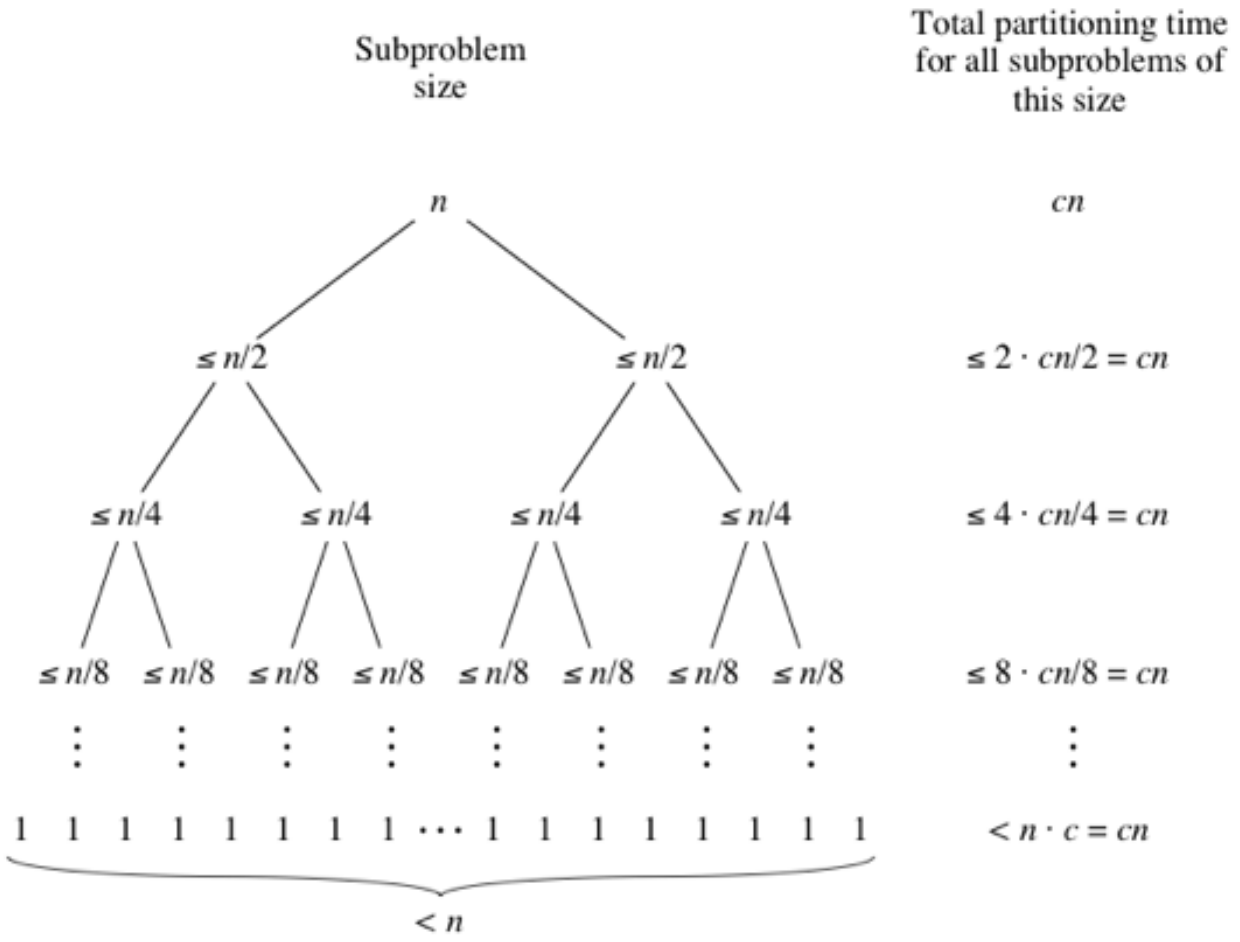
Örnek-divide & conquer: Quick sort

Best-case Analysis:

•What would be the best case?

- n Partition is perfectly balanced
- n Pivot is always in the middle

$$
\begin{aligned}
T(N) &= 2T(N/2) + cN \\
\frac{T(N)}{N} &= \frac{T(N/2)}{N/2} + c \\
\frac{T(N/2)}{N/2} &= \frac{T(N/4)}{N/4} + c \\
\frac{T(N/4)}{N/4} &= \frac{T(N/8)}{N/8} + c \\
&\vdots \\
\frac{T(2)}{2} &= \frac{T(1)}{1} + c \\
\frac{T(N)}{N} &= \frac{T(1)}{1} + c \log N \\
T(N) &= cN \log N + N = O(N \log N)
\end{aligned}
$$



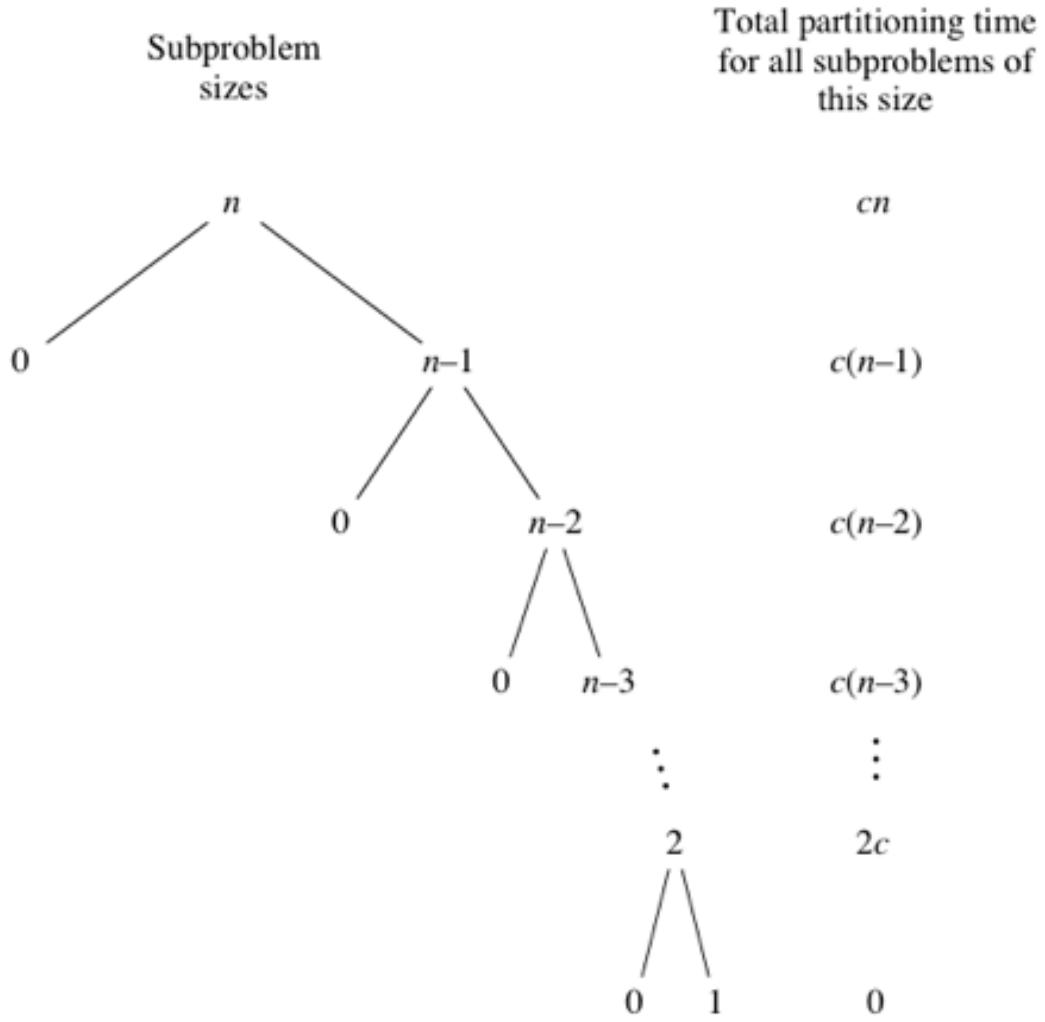
Örnek-divide & conquer: Quick sort

Worst-Case Analysis:

What would be the worst case?

- The pivot may be the smallest element, constantly
- The pivot may be the biggest element, constantly
- Partition is always unbalanced

$$
\begin{aligned}
T(N) &= T(N-1) + cN \\
T(N-1) &= T(N-2) + c(N-1) \\
T(N-2) &= T(N-3) + c(N-2) \\
&\vdots \\
T(2) &= T(1) + c(2) \\
T(N) &= T(1) + c\sum_{i=2}^{N} i = O(N^2)
\end{aligned}
$$



Örnek-divide & conquer: Quick sort

Ortalama durum

For any pivot position i; $i \in \{0, \ldots, n-1\}$:

- Time for partitioning an array : cn
- The head and tail subarrays contain i and $n-1-i$ items, respectively: $T(n) = cn + T(i) + T(n-1-i)$

Average running time for sorting (a more complex recurrence):

$$T(n) = \frac{1}{n} \sum_{i=0}^{n-1} (T(i) + T(n-1-i) + cn)$$

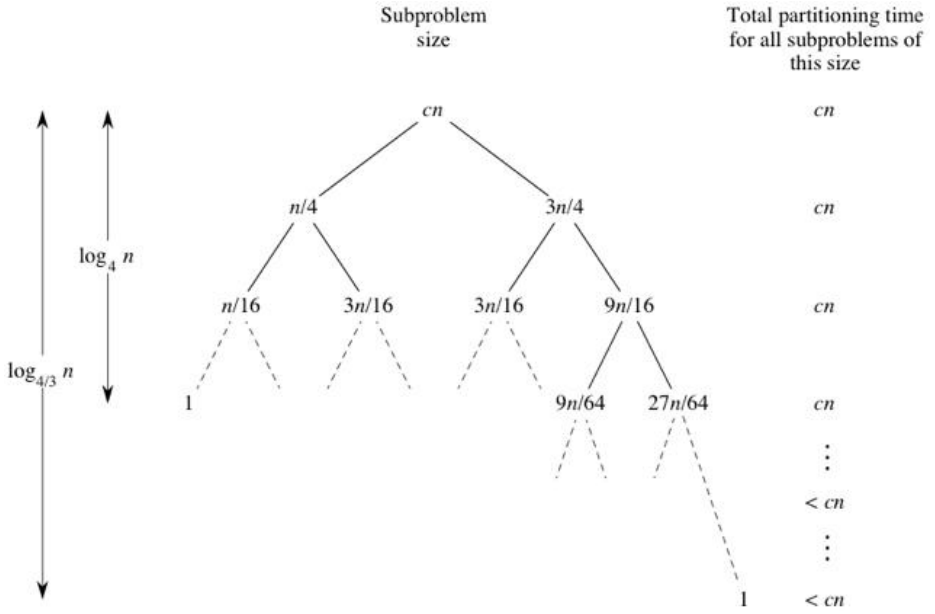
$$= \frac{2}{n} (T(0) + T(1) + \ldots + T(n-2) + T(n-1)) + cn, \text{ or}$$

$$nT(n) = 2 (T(0) + T(1) + \ldots + T(n-2) + T(n-1)) + cn^2$$

$$(n-1)T(n-1) = 2 (T(0) + T(1) + \ldots + T(n-2)) + c(n-1)^2$$

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2cn - c \approx 2T(n-1) + 2cn$$

Thus, $nT(n) \approx (n+1)T(n-1) + 2cn$, or $\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2c}{n+1}$



Örnek-divide & conquer: Quick sort

"Telescoping" $\frac{T(n)}{n+1} - \frac{T(n-1)}{n} = \frac{2c}{n+1}$ to get the explicit form:

$$\frac{T(n)}{n+1} + \frac{T(n-1)}{n} + \frac{T(n-2)}{n-1} + \ldots + \frac{T(2)}{3} + \frac{T(1)}{2}$$

$$- \frac{T(n-1)}{n} - \frac{T(n-2)}{n-1} - \ldots - \frac{T(2)}{3} - \frac{T(1)}{2} - \frac{T(0)}{1}$$

$$= \frac{2c}{n+1} + \frac{2c}{n} + \ldots + \frac{2c}{3} + \frac{2c}{2}, \text{ or}$$

$$\frac{T(n)}{n+1} = \frac{T(0)}{1} + 2c\left(\frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n} + \frac{1}{n+1}\right) \approx 2c(H_{n+1} - 1) \approx c' \log n$$

($H_n = 1 + \frac{1}{2} + \frac{1}{3} + \ldots + \frac{1}{n} \approx \ln n + 0.577$ is the nth harmonic number).

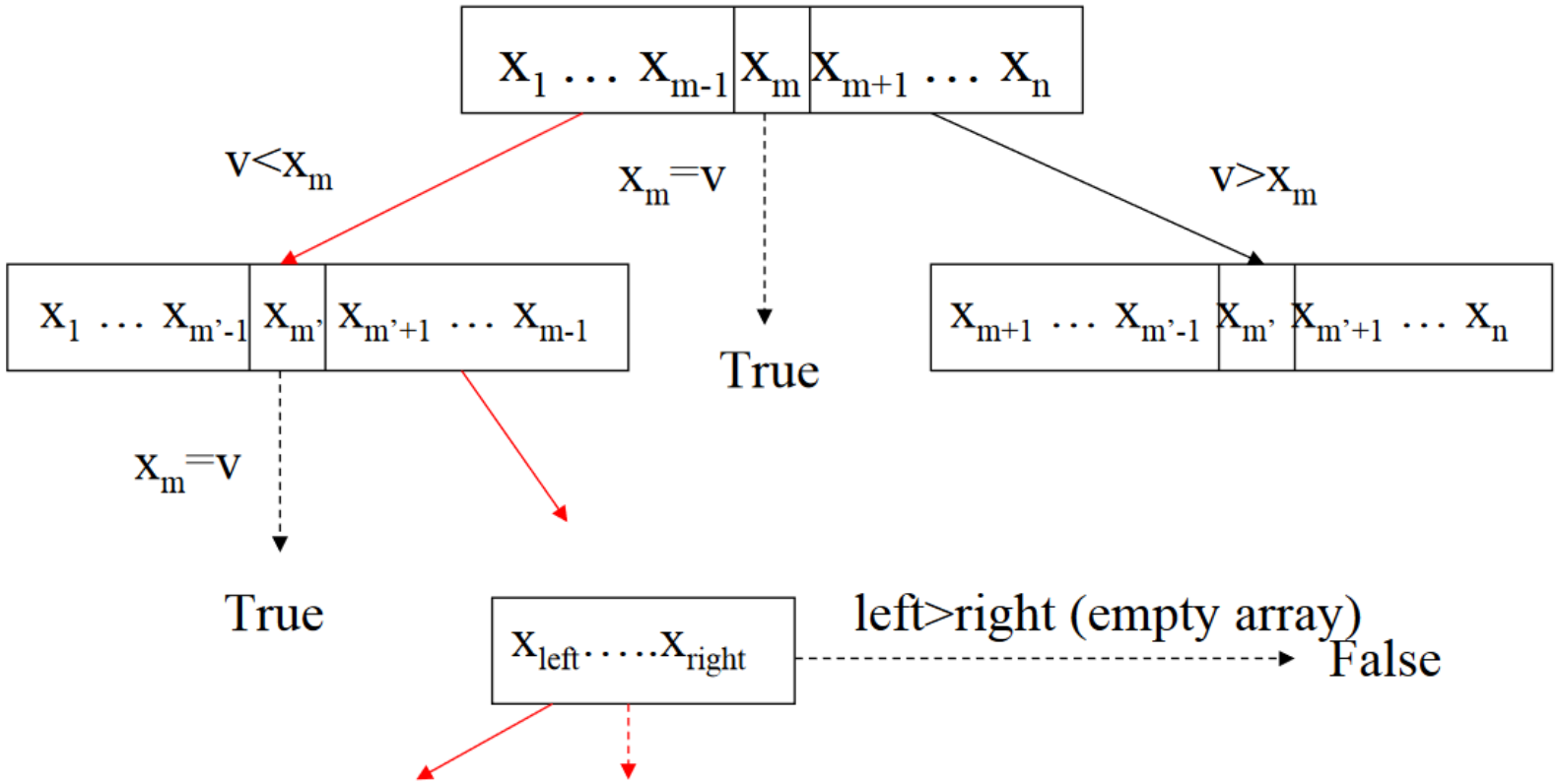
Therefore, $T(n) \approx c'(n+1)\log n \in \Theta(n \log n)$.

Decrease and Conquer

- Basic idea: exploit the relationship between the solution of a given instance of a problem and the solution of a smaller instance of the same problem. By reducing successively the problem's dimension we eventually arrive to a particular case which can be solved directly.
  Motivation:
  - such an approach could lead us to an algorithm which is more efficient than a brute force algorithm
  - sometimes it is easier to describe the solution of a problem by referring to the solution of a smaller problem than to describe explicitly the solution

Örnek–decrease&conquer: binary search

Check if a given value, v, is an element of an increasingly sorted array,
x[1..n] (x[i]<=x[i+1])



Örnek–decrease&conquer: binary search

Recursive variant:

```
binsearch(x[left..right],v)
IF left>right THEN RETURN False
ELSE
    m ←(left+right) DIV 2
    IF v=x[m] THEN RETURN True
    ELSE
        IF v<x[m]
         THEN RETURN binsearch(x[left..m-1],v)
          ELSE RETURN binsearch(x[m+1..right],v)
        ENDIF
   ENDIF
ENDIF
```

Remarks:

n_c=0

k=2

Only one of the two subproblems is solved

This is rather a decrease & conquer approach

Örnek–decrease&conquer: binary search

Efficiency:

Worst case analysis ($n=2^m$)

$$T(n)=\begin{cases}1 & n=1\\ T(n/2)+1 & n>1\end{cases}$$

$T(n)=T(n/2)+1$

$T(n/2)=T(n/4)+1$

…

$T(2)=T(1)+1$

$T(1)=1$

$T(n)=\lg n+1$ $O(\lg n)$

Örnek–decrease&conquer: binary search

Second iterative variant:

```
binsearch(x[1..n],v)
  left ← 1
  right ← n
  WHILE left<right DO
     m ←(left+right) DIV 2
     IF v<=x[m]
            THEN right ← m
             ELSE left ← m+1
      ENDIF / ENDWHILE
  IF x[left]=v THEN RETURN True
                ELSE RETURN  False
  ENDIF
```

Correctness

Precondition: n>=1

Postcondition:

"returns True if v is in x[1..n] and
False otherwise"

Loop invariant: "if v is in x[1..n]
then it is in x[left..right]"

(i) left=1, right=n => the loop invariant is true

(ii) It remains true after the execution of the loop body

(iii) when right=left it implies the postcondition

Örnek–decrease&conquer: x^n hesabı

Example. Let us consider the problem of computing x^n for $n=2^m$, $m>=1$

Since

$$x^{2\wedge m} = \begin{cases} x*x & \text{if } m=1 \\ x^{2\wedge(m-1)}*x^{2\wedge(m-1)} & \text{if } m>1 \end{cases}$$

It follows that we can compute $x^{2\wedge m}$ by computing:

$m=1 \Rightarrow p:=x*x=x^2$

$m=2 \Rightarrow p:=p*p=x^2*x^2=x^4$

$m=3 \Rightarrow p:=p*p=x^4*x^4=x^8$

....

```
Power2(x,m)
    p ← x*x
    FOR i ← 1,m-1 DO
        p ← p*p
    ENDFOR
    RETURN p
```

Bottom up approach
(start with the smallest
instance of the problem)

Analysis:

a) Correctness

Loop invariant: $p=x^{2^i}$

b) Complexity

(i) problem size: m

(ii) dominant operation: *

$T(m) = m$

Remark:

$m=\lg n$

Örnek–decrease&conquer: x^n hesabı

This idea can be extended to the case of an arbitrary value for n:

$$x\text{^}n = \begin{cases} x*x & \text{if } n=2 \\ x^{n/2}*x^{n/2} & \text{if } n>2,\ n \text{ is even} \\ x^{(n-1)/2}*x^{(n-1)/2}*x & \text{if } n>2,\ n \text{ is odd} \end{cases}$$

```
power5(x,n)
    IF  n=1 THEN RETURN x
    ELSE
        IF  n=2 THEN RETURN x*x
        ELSE
            p←power5(x,n DIV 2)
            IF n MOD 2=0 THEN  RETURN p*p
                         ELSE   RETURN p*p*x
    ENDIF ENDIF ENDIF
```

Exponentiation by Squaring

The problem: Compute a^n where n is a nonnegative integer

The problem can be solved by applying recursively the formulas:

For even values of n

$$a^n = (a^{n/2})^2 \text{ if } n > 0 \text{ and } a^0 = 1$$

For odd values of n

$$a^n = (a^{(n-1)/2})^2 a$$

Recurrence: $M(n) = M(\lfloor n/2 \rfloor) + f(n)$, where $f(n) = 1$ or 2,

$M(0) = 0$

Master Theorem: $M(n) \in \Theta(\log n) = \Theta(b)$ where $b = \lceil \log_2(n+1) \rceil$

Örnek–decrease&conquer: en büyük ortak bölen

Euclid's algorithm is based on repeated application of equality

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

Ex.: $\gcd(80,44) = \gcd(44,36) = \gcd(36, 8) = \gcd(8,4) = \gcd(4,0) = 4$

One can prove that the size, measured by the first number, decreases at least by half after two consecutive iterations.

Hence, $T(n) \in O(\log n)$

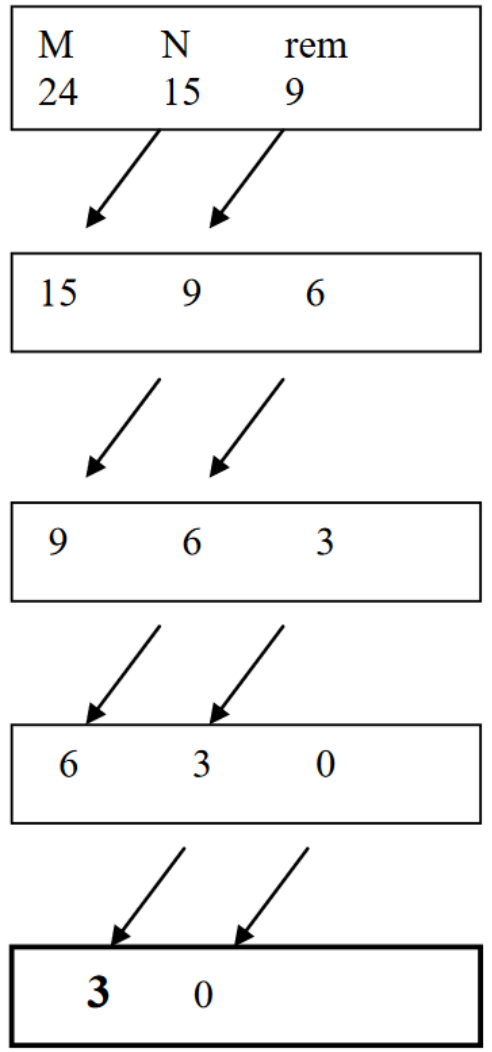
Örnek–decrease&conquer: en büyük ortak bölen
Euclid's algorithm-gcd-greatest common divisor

```
gcd (long M,long N){
      if (M%N == 0), return N;
      else return gcd(N, M%N);
}
```

```
long gcd ( long m, long n){
  long rem;
  while (n != 0){
        rem = m % n;
        m = n;
        n = rem;
  }
 return m;
}
```

Example:

M = 24, N = 15



Örnek–decrease&conquer: Insertion sort

To sort array A[0..*n*-1], sort A[0..*n*-2] recursively and then insert A[*n*-1] in its proper place among the sorted A[0..*n*-2]

Example: Sort 6, 4, 1, 8, 5

Usually implemented bottom up (nonrecursively)

6 | 4 1 8 5

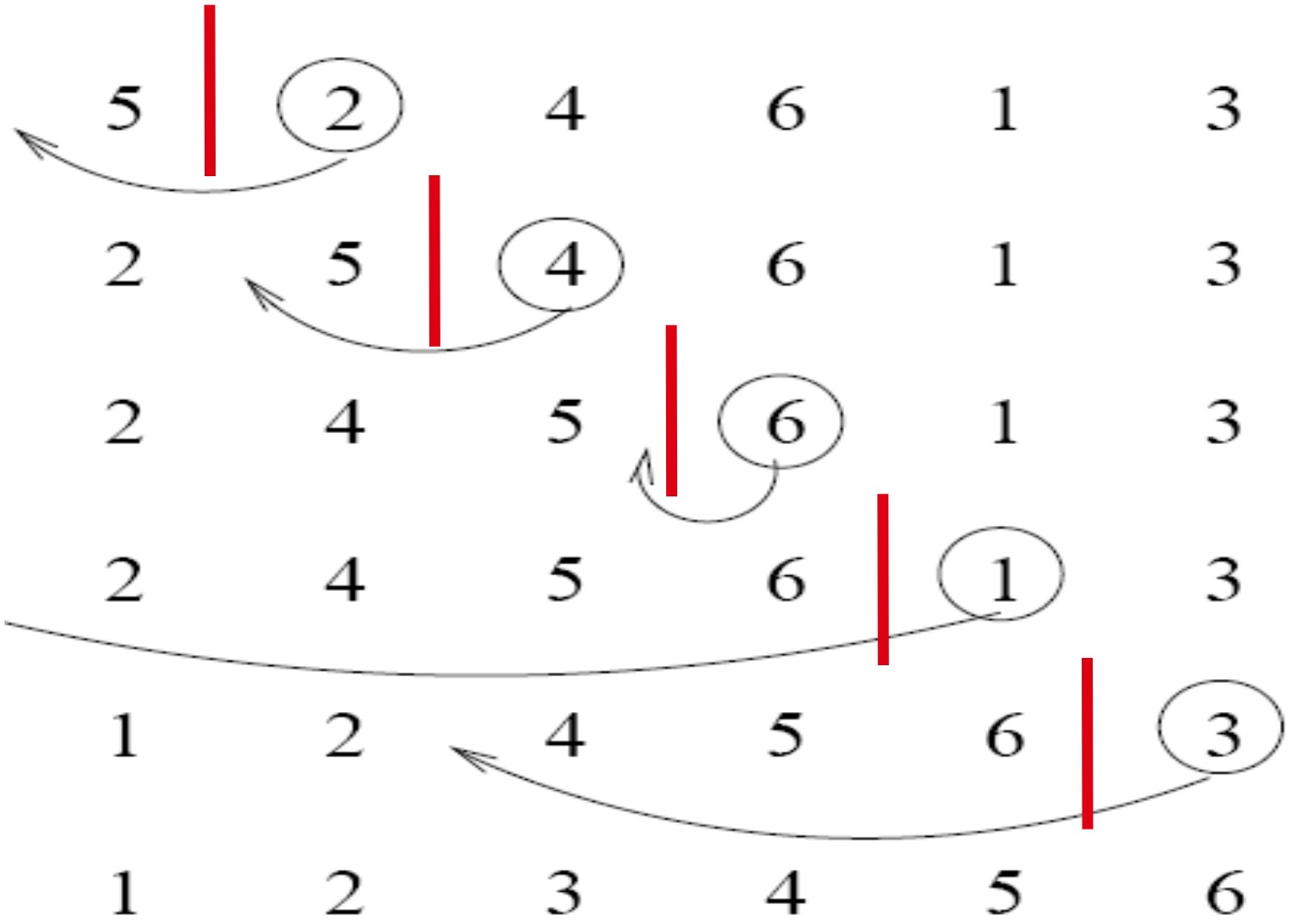
Örnek–decrease&conquer: Insertion sort

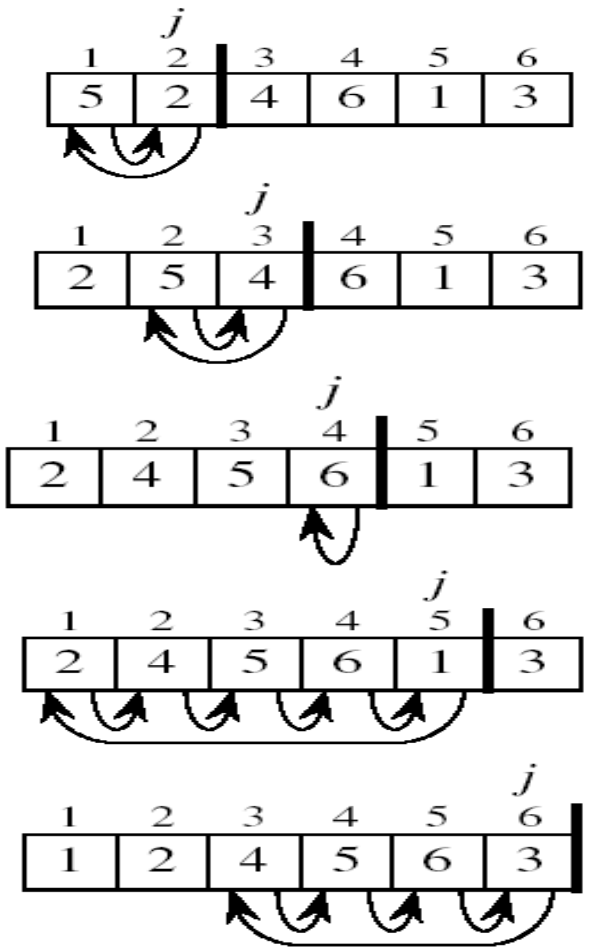
INSERTION-SORT*(A)*	cost	times
for j ← 2 **to** n	c_1	n
do key ← A[j]	c_2	n-1
Insert A[j] into the sorted sequence A[1 . . j -1]	0	n-1
i ← j - 1	c_4	n-1
while i > 0 and A[i] > key	c_5	$\sum_{j=2}^{n} t_j$
do A[i + 1] ← A[i]	c_6	$\sum_{j=2}^{n} (t_j - 1)$
i ← i – 1	c_7	$\sum_{j=2}^{n} (t_j - 1)$
A[i + 1] ← key	c_8	n-1

t_j: # of times the while statement is executed at iteration j

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^{n} t_j + c_6 \sum_{j=2}^{n} (t_j - 1) + c_7 \sum_{j=2}^{n} (t_j - 1) + c_8(n-1)$$

Örnek–decrease&conquer: Insertion sort





Best Case Analysis

- The array is already sorted

  "**while** i > 0 and A[i] > key"

  - $A[i] \leq key$ upon the first time the **while** loop test is run (when $i = j - 1$)
  - $t_j = 1$

- $T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) = (c_1 + c_2 + c_4 + c_5 + c_8)n + (c_2 + c_4 + c_5 + c_8)$

  $= an + b = \Theta(n)$

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\sum_{j=2}^{n} t_j + c_6\sum_{j=2}^{n}(t_j - 1) + c_7\sum_{j=2}^{n}(t_j - 1) + c_8(n-1)$$

Worst Case Analysis

- The array is in reverse sorted order — **"while** i > 0 and A[i] > key"
  - Always A[i] > key in **while** loop test
  - Have to compare key with all elements to the left of the j-th position ⇒ compare with j-1 elements ⇒ $t_j = j$

using $\sum_{j=1}^{n} j = \frac{n(n+1)}{2} \Rightarrow \sum_{j=2}^{n} j = \frac{n(n+1)}{2} - 1 \Rightarrow \sum_{j=2}^{n} (j-1) = \frac{n(n-1)}{2}$ we have:

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + c_6\frac{n(n-1)}{2} + c_7\frac{n(n-1)}{2} + c_8(n-1)$$

$$= an^2 + bn + c$$ a quadratic function of n

- $T(n) = \Theta(n^2)$ order of growth in n^2

$$T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5\sum_{j=2}^{n} t_j + c_6\sum_{j=2}^{n}(t_j - 1) + c_7\sum_{j=2}^{n}(t_j - 1) + c_8(n-1)$$

Comparisons and Exchanges in Insertion Sort

INSERTION-SORT*(A)*	cost	times
for j ← 2 **to** n	c_1	n
do key ← A[j]	c_2	n-1
Insert A[j] into the sorted sequence A[1 . . j -1]	0	n-1
i ← j - 1	c_4	n-1
while i > 0 and A[i] > key	c_5	$\sum_{j=2}^{n} t_j$
do A[i + 1] ← A[i]	c_6	$\sum_{j=2}^{n} (t_j - 1)$
i ← i – 1	c_7	$\sum_{j=2}^{n} (t_j - 1)$
A[i + 1] ← key	c_8	n-1

≈ $n^2/2$ comparisons

≈ $n^2/2$ exchanges

Kaynaklar

- https://en.wikipedia.org/wiki/Divide_and_conquer_algorithm
- http://gabrielistrate.weebly.com/uploads/2/5/2/6/2526487/curs9.pdf
- https://courses.engr.illinois.edu/cs473/sp2011/Lectures/05_notes.pdf
- http://faculty.simpson.edu/lydia.sinapova/www/cmsc250/LN250_Levitin/PDF/L06-DecreaseConquer12.pdf
- http://faculty.simpson.edu/lydia.sinapova/www/cmsc250/LN250_Levitin/L05-BruteForce.htm
- http://sweet.ua.pt/jmadeira/daa/DAA_02_Brute_Force.pdf
- http://www.programmingboss.com/2016/02/quick-sort-algorithm-implementation-in.html
- https://tr.khanacademy.org/computing/computer-science/algorithms/quick-sort/a/analysis-of-quicksort
- http://www.cs.princeton.edu/courses/archive/spr07/cos226/lectures/04MergeQuick.pdf